

Avanços característicos de AUV com o ROV BlueROV2

Henrique Abrantes

2025

Orientador: Prof. Pedro Teodoro

Índice

1	Introdução ao ROV	2
2	Cuidados a ter	3
2.1	Tipo de Baterias e Cuidados Associados	3
2.1.1	Cuidados a ter com Baterias LiPo	3
2.2	Cuidados na Operação do BlueROV2	3
2.3	Manutenção Preventiva e Corretiva	3
2.3.1	Manutenção Periódica (mensal ou a cada 10 mergulhos)	3
2.4	Pre-Dive Checklist	3
3	Descrição do trabalho desenvolvido	5
3.1	Métodos de comunicação	5
3.1.1	MAVLink Router	6
3.1.2	Passos de configuração / Manual de funcionamento	6
3.2	Análise de telemetria	7
3.3	Controlador PID (Proporcional integral derivativo)	8
3.3.1	Ação proporcional	8
3.3.2	Ação integral	8
3.3.3	Ação derivativa	8
3.4	Resultados e Conclusões PID	9
3.5	Navegação autónoma	10
3.6	Resultados da navegação autónoma	11
3.7	Visão (Processamento de Imagem)	12
3.7.1	Análise da imagem	12
3.7.2	Normalização HSV e Máscaras	12
3.7.3	Operações morfológicas	12
3.8	Resultados e Conclusões Visão	13
4	Conclusão	15

Coordenador de Projeto Henrique Abrantes
 Professor Pedro Teodoro
 Equipa ROV

1 Introdução ao ROV

Desenvolvido pela BlueRobotics o BlueROV2 é um drone subaquático adquirido pela faculdade. Demonstra ser uma plataforma para começar a aprender e aplicar tecnologia marítima. O artigo colocado nas referências 'Survey on advances on terrain based navigation for autonomous underwater vehicles'[MM17] demonstrou ser relevante para introduzir a tecnologia marítima nas fases iniciais da cadeira e do projeto encontra-se disponível também uma revisão crítica feita por Henrique Abrantes [Abr25a].

Desde desenvolver software para INS autónomo, visão (processamento de imagem) e controlo este relatório visa partilhar em detalhe o trabalho realizado, desafios encontrados e conhecimentos adquiridos na cadeira de Robótica Marítima com o BlueROV2. Com várias capacidades para pesquisa demonstra ser uma ferramenta versátil para adquirir conhecimentos essenciais na área da robótica marítima e consequentemente fazer avanços tecnológicos. Dispõe os seguintes sensores.

Sensor	Tipo/Função
Ping Sonar Altimeter and Echosounder	Medição de profundidade/distância
Ping360 Scanning Imaging Sonar	Sonar de varrimento em 360°
Newton Subsea Gripper	Garra para manipulação
Sensores IMUs	Movimento e orientação (inercial)
Sensor de pressão	Medição precisa de profundidade
Câmara	Visão subaquática em tempo real

Table 1: Sensores do BlueROV2

Para saber mais sobre os como funcionam os sensores e como o ROV se aplica na robótica marítima. Ver ao artigo [MM17] e especificamente sobre o ROV a revisão crítica [Abr25a].



Figure 1: Caixa ROV [Dan25]



Figure 2: ROV [Dan25]

2 Cuidados a ter

Para manusear o ROV é preciso ter muitos cuidados dentro e fora de água, antes e depois de ser utilizado. Esta parte do relatório pretende fornecer um guia prático e seguro para a operação do veículo subaquático remotamente operado (ROV) BlueROV2 da BlueRobotics. Serão abordadas as melhores práticas relativas ao uso de baterias, cuidados operacionais, manutenção preventiva e corretiva, bem como um checklist de preparação para mergulho (pre-dive checklist).

2.1 Tipo de Baterias e Cuidados Associados

O BlueROV2 utiliza baterias de polímero de lítio (LiPo), geralmente do tipo 4S ou 6S com capacidade entre 10.000 a 18.000 mAh. [Lia22]

2.1.1 Cuidados a ter com Baterias LiPo

É necessário verificar visualmente se as baterias têm sinais de inchaço, danos físicos ou fugas antes de cada utilização. No carregamento de baterias é sempre preciso supervisão e é apenas permitido utilizar carregadores LiPo balanceados (como o disponível na caixa do ROV). Se a bateria não for usada durante longos períodos de tempo (como, por exemplo, o fim do ano letivo) a bateria tem que estar com cerca de 50% de carga e armazenada em sacos LiPo à prova de fogo. Na utilização das baterias verifique sempre se está firmemente presa e se os cabos não estão em risco de serem danificados.

2.2 Cuidados na Operação do BlueROV2

É necessário manter sempre uma distância segura entre obstáculos dentro e fora de água, para evitar eventuais colisões. Evitar contacto com o fundo (da piscina ou do mar). Verifique sempre se o cabo tether está livre de enrolamentos e evite tensão excessiva. Evitar operar o ROV nas temperaturas fora da faixa recomendada (0-40°C). [Blu24a]

2.3 Manutenção Preventiva e Corretiva

Após cada missão é necessário lavar o ROV com água doce para o preservar (principalmente os propulsores), passar com água e girar com a mão cada propulsor do ROV. Caso deixe o ROV dentro da caixa com ele ligeiramente molhado, mantenha a caixa aberta. Para manutenção preventiva verificar o estado dos O-rings (têm que estar lubrificados) e lubrificar com silicone apropriado. [Blu24b]

2.3.1 Manutenção Periódica (mensal ou a cada 10 mergulhos)

É necessário abrir os compartimentos e inspecionar condensação ou infiltração. Substituir os O-rings gastos e testar o funcionamento dos motores (thrust balance). Atualizar firmware e software via BlueOS.

2.4 Pre-Dive Checklist

Segue a lista de verificação recomendada pela BlueRobotics para garantir a segurança da operação [Blu24d].

1. Verificação Visual Geral do ROV

- Ausência de danos físicos.
- Thrusters limpos e desobstruídos.
- Tether devidamente conectado e sem danos.

2. Integridade de Vedações

- O-rings em bom estado e lubrificados.
- Compartimentos fechados e bem selados. (teste de vácuo)

3. Sistema Elétrico e Eletrónico

- Bateria carregada e corretamente instalada.
- Verificar tensão da bateria via interface BlueOS.
- Comunicação estabelecida com BlueOS / QGroundControl.
- Sensores operacionais (IMU, barómetro, altímetro, sonar).

4. Testes em Terra

- Testar os thrusters para averiguar se respondem corretamente aos comandos.
- Verificar se a Câmara transmite vídeo com qualidade.
- Funcionamento das luzes e outros periféricos funcionais.

5. Planeamento da Missão

- Verificar as condições meteorológicas e marítimas.
- (caso aplicável) Trajeto e profundidade máxima definidos.
- Validar equipamento de recuperação de emergência disponível.

A operação segura e eficiente do BlueROV2 exige uma abordagem sistemática e disciplinada. A manutenção regular, a preparação cuidadosa e o conhecimento dos limites do equipamento são fundamentais para prolongar a vida útil do ROV e garantir missões bem-sucedidas. [Blu24c; Blu24a]

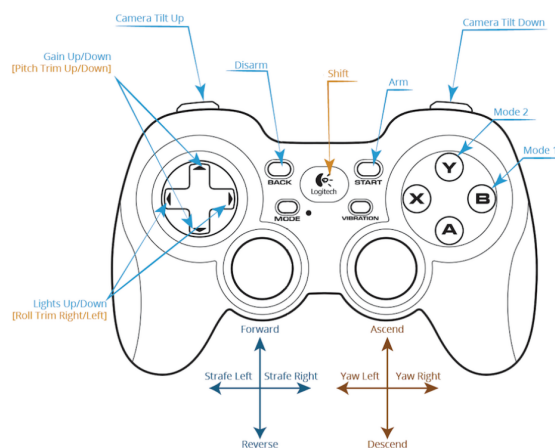


Figure 3: Funcionalidades padrão comando [Blu24a]

Nota: Este documento pode ser atualizado conforme novas recomendações da BlueRobotics ou alterações no sistema do BlueROV2. Consulte sempre a documentação oficial em: <https://bluerobotics.com> [Blu24c; Blu24a]

3 Descrição do trabalho desenvolvido

Antes de explicar em detalhe o algoritmo e trabalho desenvolvido é necessário explicar de maneira breve o software presente no ROV.

O BlueOS é uma plataforma moderna baseada em web, desenvolvida especificamente para veículos subaquáticos como o BlueROV2. É executado a bordo do ROV (normalmente num Raspberry Pi) e oferece uma interface acessível via navegador que permite a configuração e monitorização do sistema em tempo real.

Funcionalidades principais incluem:

- Gestão de dispositivos: deteção e configuração de sensores (IMU, sonar, GPS, etc.).
- Atualizações e extensões: sistema de plugins e atualizações automáticas que facilitam a manutenção.
- Integração com o QGroundControl via MAVLink, facilitando a comunicação entre o ROV e o operador.
- Acesso remoto: suporte para acesso remoto ao sistema, útil para testes e manutenção fora do local.

O BlueOS é particularmente relevante para quem pretende integrar novos sensores ou algoritmos personalizados no ROV, pois permite fácil acesso aos dados e à estrutura de comunicação do sistema. Foi utilizado para criar ativar um MAVLink endpoint para podermos aceder via script Python o ROV e utilizado para saber quais os IPs/porta série dos sensores e da câmara.

O QGroundControl (QGC) é uma estação de controlo de solo cross-platform utilizada no computador de bordo do ROV e usa o protocolo MAVLink. Este utilizado para planejar, monitorizar e controlar as missões do ROV em tempo real.[Pro24]

Principais características:

- Interface gráfica rica e intuitiva, com visualização de dados em tempo real.
- Planeamento de missões autónomas, com definição de pontos de passagem (waypoints) e ações associadas.
- Calibração de sensores como IMUs, magnetómetros e barómetros.
- Visualização de telemetria, mapas, gráficos e mensagens do sistema.

As etapas/projetos que o professor Pedro Teodoro nos propôs foram:

- **Controlo:** Manter a profundidade via controlo PID; Percorrer de um lado ao outro na piscina de maneira autónoma mantendo a profundidade.
- **Visão:** Utilizar a câmara para o ROV passar dentro de um arco de maneira autónoma.

3.1 Métodos de comunicação

Para conseguir alcançar os objetivos propostos é necessário garantir a comunicação ao ROV sem ser a utilizar o software QGroundControl e o BlueOS. Os métodos de comunicação utilizados foram:

Método de comunicação	Sensor
Comunicação Série MAVLink	Ping360 Scanning Imaging Sonar, Ping Sonar Altimeter and Echosounder, Câmara Sensores IMUs, Sensores de pressão

Table 2: Métodos de comunicação

Comunicação série - Para descobrir qual o endereço dos sensores utilizou-se o PingViewer, pois ele faz uma busca da rede e dá o endereço e a porta dos sensores Ping disponíveis, também se pode usar o software BlueOS na aba dos sensores.

PyMavLink - Utilizado para estabelecer uma conexão com o ROV via Script de Python. PyMavLink é uma biblioteca de processamento de mensagens MAVLink de baixo nível e de uso geral.

Para utilizar MAVLink primeiro ativou-se no BlueOS um MAVLink-endpoint. O MAVLink-endpoint utilizado foi o GCSServerLink (Ground Control Station Server Link). Trata-se de um endpoint UDP que permite um GCS (Ground Control Station) conectar ao veículo via protocolo MAVLink. O GCS envia comandos para o ROV que solicitam dados de telemetria, recebendo em troca os dados solicitados, estados do sistema, dados dos sensores puro ou dados dos sensores previamente tratados. Por ser um Endpoint UDP foi necessário validar os dados antes de serem enviados para o algoritmo.

3.1.1 MAVLink Router

O MAVLink Router é uma aplicação que distribui mensagens MAVLink, entre múltiplos pontos de conexão. Permite que pacotes MAVLink sejam distribuídos para um único destino ou para múltiplos endpoints, objetivo.

Tivemos de utilizar o MAVLink Router porque o QGroundControl fechava o único endpoint disponível, para comunicação com o ROV. Para configurá-lo tivemos que instalar o MAVLink router no computador do ROV que serviu de router entre o QGroundControl e os Scripts de Python.

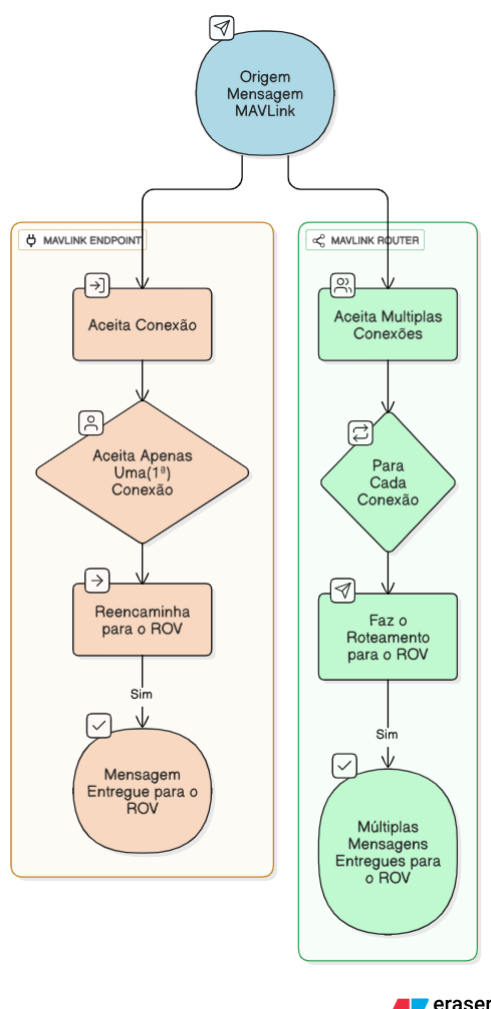


Figure 4: MAVLink endpoint vs MAVLink Router

3.1.2 Passos de configuração / Manual de funcionamento

Nesta secção vamos abordar os comandos essenciais/utilizados, os IPs e a configuração geral realizada no MAVLink router e no computador do Host. Para manual de funcionamento/comandos to MAVLink encontra-se disponível no site do MAVLink [Con25]

Inicialmente foi configurado um MAVLink endpoint no BlueOS com o IP 0.0.0.0 : 14550. Para utilizar o MAVLink Router tivemos que o instalar no WSL (Windows Subsystem for Linux). Para o configurar criámos o ficheiro *main.conf* onde colocamos as configurações pretendidas. Ficheiro *main.conf*,

```
[General]
ReportStats=false
MavlinkDialect=auto
Log=/tmp/logs
LogMode=always

[UdpEndpoint blue_rov]
Mode=Normal
Address=0.0.0.0
Port=14550

[UdpEndpoint qgroundcontrol]
Mode=Normal
Address=0.0.0.0
Port=14551

[UdpEndpoint python_script]
Mode=Normal
Address=0.0.0.0
Port=14552
```

Porquê 0.0.0.0 e não outro IP? O IP 0.0.0.0 ouve de qualquer interface. Não à problemas de segurança envolvidos porque estamos num ambiente isolado. Para colocar tudo a funcionar em sintonia configurámos uma ponte entre o computador host e o WSL.

Inicialmente para colocar o MAVLink Router a funcionar era preciso fazer este comando, *./build/src/mavlink-routerd -c ./main.conf*. Para facilitar o uso configurei-o como serviço configurando o ficheiro *mavlink-router.service*,

```
[Unit]
Description=MAVLink Router
After=network.target
[Service]
ExecStart=/home/tespria/mavlink-router-master/build/mavlink-routerd -c /home/tespria/m
Restart=on-failure
User=tespria
[Install]
WantedBy=multi-user.target
```

Localização do ficheiro: */etc/systemd/system/mavlink-router.service*

Para colocar o MAVLink Router a funcionar é necessário executar os seguintes comandos: *sudo systemctl daemon-reload, sudo systemctl restart mavlink-router, sudo systemctl status mavlink-router*.

Para verificar a configuração do MAVLink/analisar o tráfego foi utilizado o WireShark no Windows e o mavproxy no Ubuntu (WSL). *./local/bin/mavproxy.py --master = udp : 127.0.0.1 : PORTA*

Ao colocar o MAVLink Router como serviço foi possível analisar melhor o seu funcionamento, um método de funcionamento otimizado.

3.2 Análise de telemetria

A análise de telemetria dos sensores foi essencial para saber se é necessário aplicar um filtro nos dados que estão a ser recebidos pelos sensores. O sensor sob análise é o Ping Sonar Altimeter and Echosounder. Para fazer a análise desse sensor foi necessário manter a profundidade do ROV, foi utilizado para o efeito um controlador PID.

3.3 Controlador PID (Proporcional integral derivativo)

“O controlador PID desempenha um papel fundamental na estabilidade e precisão de sistemas automatizados como o BlueROV2. Este tipo de controlador é essencial para garantir que o ROV consiga manter a sua profundidade, orientação e velocidade coerente, mesmo em ambientes irregulares e imprevisíveis, como o fundo do mar. O PID permite que o ROV reaja automaticamente a estas mudanças, ajustando os motores de forma contínua e precisa, sem necessidade de intervenção humana constante. Com os parâmetros corretamente ajustados, o sistema é capaz de manter o equilíbrio e a direção desejada, o que é crucial para operações de inspeção, recolha de dados ou tarefas técnicas em ambientes subaquáticos.” [Fra25]

Um controlador PID, na área de controlo, é um algoritmo de feedback muito usado pela sua versatilidade, facilidade de implementação e resiliência durante a sua utilização. Combina três tipos de ações, a ação proporcional (K_p), ação integral (K_i) e ação derivativa (K_d). Para alcançar melhor desempenho do controlador PID foi necessário ajustar valores. Esse ajuste pode ser feito através do método tentativa e erro até o controlador alcançar um bom comportamento, ou, através de inteligência artificial ou algoritmos de otimização para encontrar os melhores parâmetros durante a utilização do controlador, criando um ambiente dinâmico. Apesar de ser versátil e fácil de implementar existem outros desafios associados, como, por exemplo, ruídos no sistema (que será abordado a seguir).[25b]

Variações do controlador PID, [Fra25]

- **PI (Proporcional + Integral):** utilizado quando o sistema é particularmente sensível a ruído.
- **PD (Proporcional + Derivativo):** indicado para sistemas que já são naturalmente estáveis.
- **PID adaptativo:** ajusta automaticamente os seus parâmetros ao longo do tempo.
- **PID com inteligência artificial:** recorre a redes neuronais ou lógica fuzzy para tomar decisões mais inteligentes.

Onde $u(t)$ é o sinal de saída,

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Transformada de Laplace: (s = frequência complexa)

$$L_s = K_p + K_i/s + K_d s$$

3.3.1 Ação proporcional

A ação proporcional produz um sinal de saída proporcional à amplitude do erro $e(t)$, sendo K_p a constante de proporcionalidade:

$$P_{saída} = K_p e(t)$$

3.3.2 Ação integral

A ação integral produz um sinal de saída proporcional à magnitude e à duração do erro, ou seja, ao erro acumulado. Isso fornece uma alternativa para corrigir o erro de off-set gerado pela ação proporcional, acelerando a resposta do sistema, permitindo-o chegar ao valor de referência mais rapidamente. O sinal de saída do controlador PI pode ser apresentado por:

$$I_{saída} = K_i \int_0^t e(\tau) d\tau$$

onde K_i é o ganho integral

3.3.3 Ação derivativa

A ação derivativa produz um sinal de saída proporcional à velocidade de variação do erro:

$$D_{saída} = K_d \frac{de(t)}{dt}$$

onde K_d é o ganho derivativo.

A mudança de cada parâmetro influencia a performance,

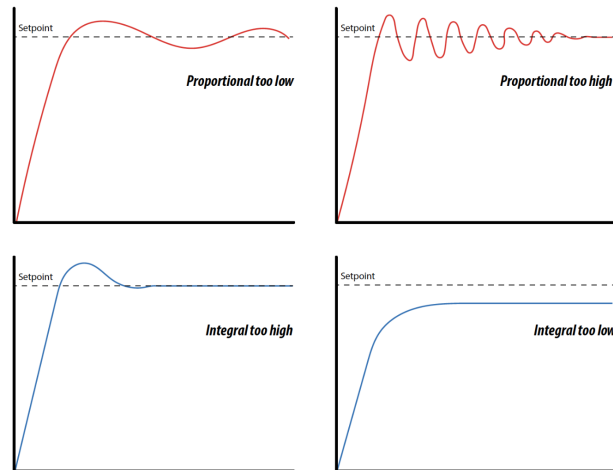


Figure 5: PID [25b]

3.4 Resultados e Conclusões PID

(Código no GitHub [Abr25d]) (Disponível um relatório que aborda em detalhe todos os testes PID realizados [Abr25b])

Nesta secção vamos analisar o comportamento do ROV bem como a telemetria obtida do PID. Os dados que vão ser analisados, correspondem ao sensor Ping1d, obtidos na piscina da faculdade.

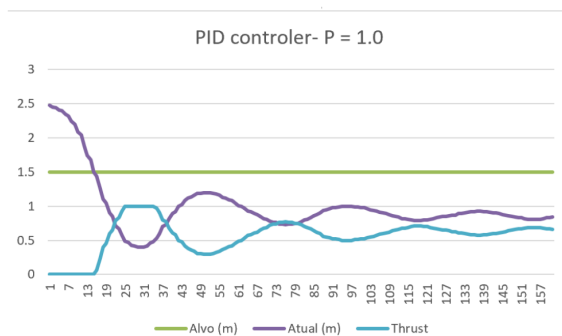


Figure 6: Primeira análise PID [Abr25b]

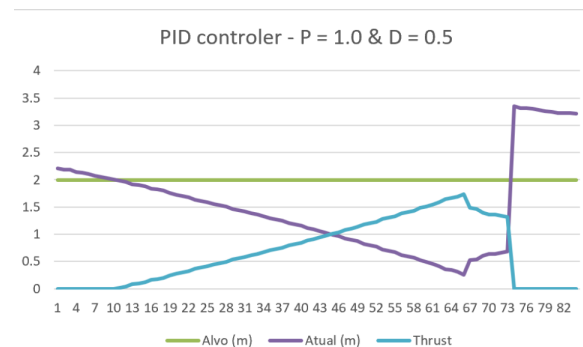


Figure 7: Segunda análise PID [Abr25b]

Podemos verificar no primeiro gráfico que o ROV chega a um nível de $thrust < 0,5$ e desce de profundidade. Convém relembrar que o sinal de controlo dos propulsores do ROV é, $u(t)$ a saída do PID, $u(t) = 0 = FullReverse$, $u(t) = 0,5 = Neutro$, $u(t) = 1 = Fullpropulsores$. O PID dá um valor entre $-\infty$ e $+\infty$ sendo $0 = neutro$. Especificamente para os propulsores verticais (eixo Z), é preciso que quando o controlador PID colocar os propulsores no neutro, o output para os propulsores serem '0,5', e não '0'. Para fazer isso é preciso que,

$$e = 0 \rightarrow u(t) = 0 \rightarrow u'(t) = 0,5$$

$$e > 0 \rightarrow u(t) > 0 \rightarrow u'(t) > 0,5$$

$$e < 0 \rightarrow u(t) < 0 \rightarrow u'(t) < 0,5$$

Sendo $u(t)$ a saída do PID e $u'(t)$ é a saída que precisa de ter para os propulsores para evitar o problema referido e analisado.

O ROV é apenas centrado em 0,5 no thruster vertical (eixo Z), no thruster do eixo X (frente/trás) e no eixo Y (lateral) é centrado em 0.

Acerca da mudança repentina de profundidade (Figura 7), deve-se ao facto de o sensor chegar muito perto do fundo da piscina e pela sua superfície lisa o sensor não consegue obter um valor correto, podemos analisar na 'Confiança' do sensor que é um dado recolhido do próprio.

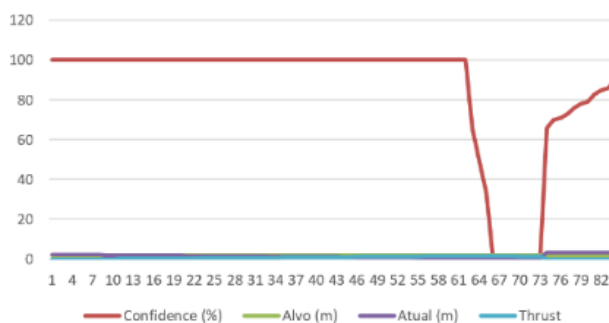


Figure 8: Análise da confiança PID [Abr25b]

Último teste PID,

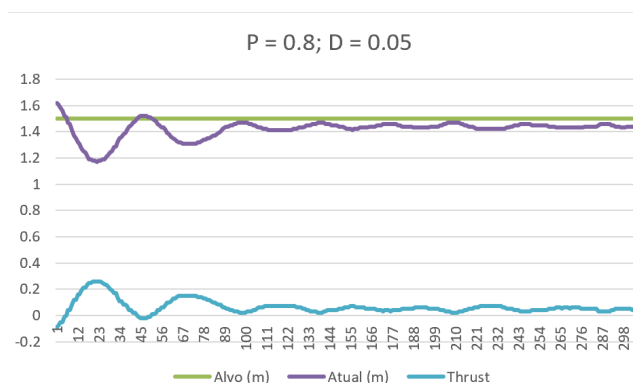


Figure 9: Última análise PID

Foram feitos ao todo '4' testes, para análise dos dados refiro ao relatório [Abr25b].

3.5 Navegação autónoma

Para o ROV percorrer a piscina de maneira autónoma, mantendo a profundidade, foi necessário utilizar controlo PID em conjunto com o sensor Ping sonar 360. A utilização deste sensor foi essencial para detetar obstáculos e evitar colisões.

Durante o funcionamento sequencial (código linear), o loop de controlo PID e o loop de aquisição do sonar partilham o mesmo fluxo de execução (o que pode consumir alguns milissegundos ou mais, dependendo da taxa de amostragem, o varrimento do Ping sonar 360 fica momentaneamente suspenso. Consequentemente, quando o processo do sonar é retomado, o ROV já se poderá ter deslocado fisicamente, o que faz com que a amostragem não corresponda com precisão à posição anterior.

O desfasamento pode introduzir gerar problemas como atrasos na reação à presença de obstáculos, risco de colisão e incoerência temporal entre a posição do ROV e os dados do sonar.

Para contornar este problema, o projeto implementa execução concorrente utilizando multithreading. Assim o loop de controlo de profundidade (PID) e o loop de varrimento do sonar correm em paralelo. Para sincronizar as decisões entre os processos paralelos, utiliza-se uma 'flag' de deteção de obstáculos e um mecanismo de 'lock' para acesso seguro. Assim que um obstáculo é detetado pelo sonar, a 'flag' é ativada, e o loop de controlo PID ajusta o movimento de evasão imediatamente.

3.6 Resultados da navegação autónoma

No teste não foi possível utilizar o sensor Ping360, porém foi possível testar o algoritmo onde o ROV anda em frente enquanto mantém a profundidade

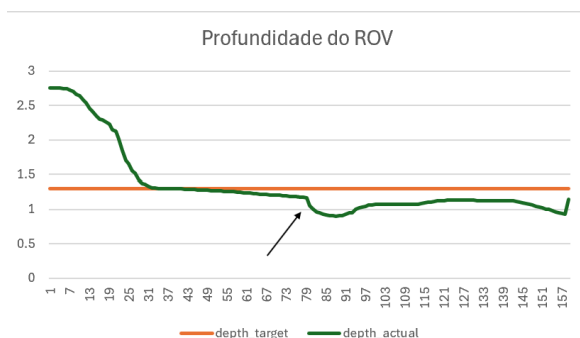


Figure 10: Profundidade do ROV

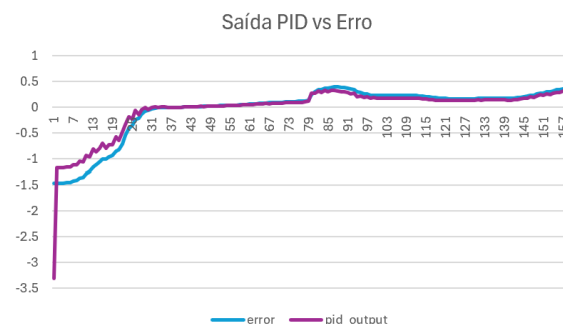


Figure 11: Erro vs Saída PID

A seta marca o momento em que o ROV passa pela rampa da piscina, onde a profundidade começa a reduzir, que consequentemente, fica mais próximo do fundo. Porém o comportamento foi o esperado e conseguimos analisa-lo na figura do erro vs saída PID que o comportamento é o esperado.

O teste foi bem sucedido e o ROV teve o comportamento esperado. Foi necessário este teste para analisar o comportamento do ROV a utilizar 2 eixos ao mesmo tempo, o x e o z

3.7 Visão (Processamento de Imagem)

O processamento de imagem foi necessário para concluir um dos objetivos do professor Pedro Teodoro.

Consistiu em receber a stream da Câmara embutida no ROV e detetar o arco vermelho na água, de seguida detetar o centroide do arco, compará-lo com o centroide da Câmara e colocar a diferença no PID, normalizar os valores e atuar os propulsores.

3.7.1 Análise da imagem

Quando recebemos a imagem do ROV temos de detetar o centroide, fazemos da seguinte maneira. A área vermelha (do arco) tem uma quantidade de pixels do objeto, com esse valor dividimos pelo valor de pixels totais e o resultado é a área desse objeto.

$$A_{\text{arco}} = \frac{\text{pixelsObjeto}}{\text{pixelsTotais}}$$

Esta conta não é utilizada no código, pois verificamos pixel a pixel e adicionamos +1 à área total do arco (processo automatizado com uma biblioteca).

Para saber as coordenadas do centroide é preciso da área do arco e uma função que pegue as coordenadas de todos os j (x) e os some ($\sum$) e divide pela área, dando o j (x) do centroide. Para saber o i (y) é fazer o mesmo, mas com todas as coordenadas do i .

$$j_{\text{centroide}} = \frac{\sum j_{\text{centroide}}}{A_{\text{arco}}}$$

$$i_{\text{centroide}} = \frac{\sum i_{\text{centroide}}}{A_{\text{arco}}}$$

A comparação entre o centroide do objeto (arco vermelho) e o centroide da Câmara é feita ao calcular a diferença entre suas coordenadas ($j_{\text{centroide objeto}}, i_{\text{centroide objeto}}$) e ($j_{\text{centroide Câmara}}, i_{\text{centroide Câmara}}$). Essa diferença representa o deslocamento necessário para alinhar o ROV ao arco.

Com a diferença calculada, o sistema chama o controlador PID, que processa o erro gerado pelo desvio e ajusta dinamicamente os comandos de movimento do ROV. O objetivo do PID é minimizar essa diferença, garantindo que o centroide da câmara coincida com o centroide do arco, permitindo uma passagem precisa dentro da estrutura subaquática.

3.7.2 Normalização HSV e Máscaras

HSV é um sistema de cores formadas pelas componentes hue (matriz), saturation (saturação) e value (valor). Este sistema define o espaço de cor da seguinte maneira: Matriz (tonalidade): Verifica o tipo de cor, abrangendo todas as cores do espectro. Saturação: Quanto menor este valor, mais o tom de cinza aparecerá na imagem. Valor (brilho): Define o brilho da cor. Todos estes valores são normalizados de 0 a 100 (%).

Porquê HSV e não RGB? No modelo HSV a matriz representa a cor pura, enquanto o valor indica o brilho, facilitando a correção de iluminação variável em ambientes subaquáticos. Ajuda também na redução de interferências pois a água absorve e dispersa luz de forma inconsistente afetando a percepção das cores, o HSV tem a saturação para compensar essa distorção. Como o HSV separa a cor e a intensidade é mais fácil identificar/destacar elementos específicos como o arco, sem ser afetado por sombras ou reflexos.

As máscaras servem para aplicar filtros e transformações em regiões específicas de uma imagem. Deste a suavização e remoção de ruído, detecção de padrões e correção de iluminação.

3.7.3 Operações morfológicas

Operações morfológicas são úteis para analisar a estrutura do arco na imagem. São baseadas na morfologia matemática e são úteis para segmentação, filtragem e análise de formas. As operações utilizadas foram as seguintes.

- **Abertura** (`cv2.MORPH_OPEN`): remove pequenos ruídos brancos da imagem binária (a máscara).
- **Fecho** (`cv2.MORPH_CLOSE`): Esta operação preenche pequenos buracos pretos dentro das regiões brancas.
- **Dilatação** (`cv2.dilate`): Expande as regiões brancas, tornando os objetos um pouco maiores. Ajuda a unir regiões próximas.

- **Erosão** (`cv2.erode`): Encolhe as regiões brancas, removendo bordas finas. Neste caso, parece ser usada para refinar novamente a máscara após a dilatação.

3.8 Resultados e Conclusões Visão

(Código no GitHub [Abr25d])

Para abrir a câmara do ROV tivemos os seguintes desafios. O Stream da câmara está a ser feito diretamente no USB na porta `\dev\dev2` ou no IP `192.168.2.1 : 14777`, porém o computador host não consegue abrir a câmara. Para o fazer foi preciso utilizar o VLC. No BlueOS temos a opção de extrair o ficheiro SDP da câmara (que é um ficheiro informativo usado para descrever sessões de transmissão de media) e o abrimos através do VLC. A biblioteca CV2 e o VLC não são compatíveis então para resolver este problema tive que converter cada frame do VLC para uma variável no código,

```
def conversion(player):
    snapshot_path = "vlc_snapshot.png"
    result = player.video_take_snapshot(0, snapshot_path, 640, 480)
    if result != 0:
        raise RuntimeError("Erro ao capturar frame do VLC.")
    frame = cv2.imread(snapshot_path)
    if frame is None:
        raise RuntimeError("Erro ao ler snapshot.")
    return frame
```

Aplicando o algoritmo abordado nos temas anteriores temos os seguintes resultados:

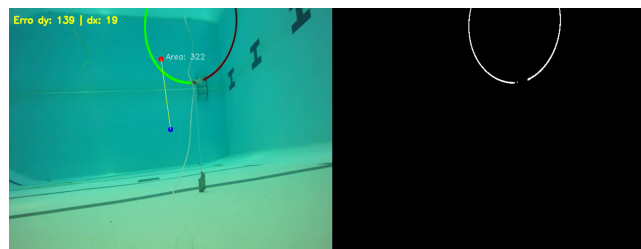


Figure 12: ROV a detetar o arco

O ROV detetou com sucesso o arco, porém, ao atuar podemos observar que o ROV fica descontrolado. A configuração PID usada foi com o $P = 1$, (**o mesmo do que o último teste PID abordado neste relatório**), logo a saída do PID estava muito agressiva. Por essas razões o ROV atuou com os propulsores para a frente e para cima ao mesmo tempo causando uma ligeira inclinação para cima, essa inclinação causou com que o ROV detetasse o teto da piscina, causando a instabilidade e o insucesso do objetivo.

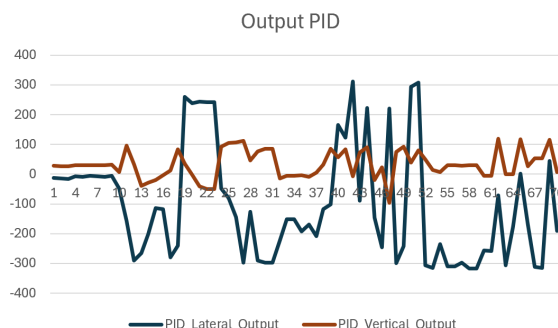


Figure 13: Saída PID

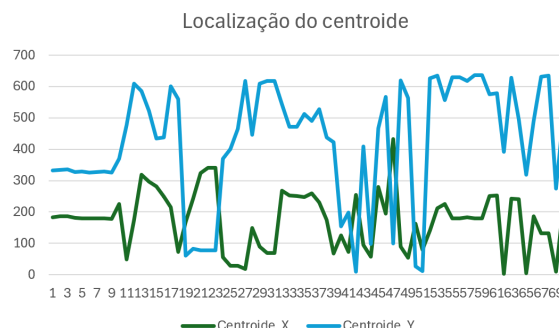


Figure 14: Localização do Centroide

Para resolver esse problema, decidiu-se separar os eixos da seguinte forma: primeiramente, deteta-se o objeto utilizando o sensor Ping360, com validação por visão; de seguida, aproxima-se o ROV até uma distância aceitável

do objeto, e com a câmara, deteta-se o centroide do objeto. O ROV irá alinhar então o eixo z . Quando estiver na profundidade correspondente ao centro do arco, o sensor Ping1d (sensor de profundidade) fará a leitura da distância ao fundo, e o algoritmo testado na secção 3.4 manterá a profundidade. Se a profundidade estiver dentro de um limite aceitável, o ROV alinhará o eixo x e, quando ambos estiverem alinhados, seguirá em frente (eixo y).

Na figura 12, o arco tem uma quebra que não está a ser corrigida pelo algoritmo. Para preencher os píxeis em falta, é necessário recorrer a operações morfológicas, já abordadas na Secção 3.5.3.

4 Conclusão

Foi-me dada a responsabilidade e a confiança de ser o técnico responsável pela manutenção do ROV, e, consequentemente na cadeira de robótica marítima, ter ficado a gerir o projeto e as diferentes equipas de desenvolvimento e de pesquisa.

Foi muito interessante ter aprendido tecnologias como TBN e navegação acústica apesar de não ter sido possível fazer o desenvolvimento por não haver os sensores adequados para a recolha dos dados necessários. Foi também muito recompensador ver as equipas a se adaptarem à mudança da direção do projeto.

Este projeto foi não só um desafio de aprendizagem e adaptação a novas tecnologias, mas também um grande desafio de gestão de equipas.

Equipa	Membros
Pesquisa	Francisca Grazina, Diogo Soares
Visão	Daniel Mendes, João Stoffel, Francisco Azevedo
Manutenção	Rafael, Eurico Martins
Comunicação	Guilherme Santos, André
Suporte Operacional	Bruno Machado, Daniel Mendes, Stoffel
TBN, Nav. Acústica	Daniel Mendes, João Stoffel
Desenvolvimento e Implementação	Tiago Ferreira, Henrique Abrantes

Table 3: Lista de Equipas

Membros
Francisca Grazina, Diogo Soares
João Stoffel, Francisco Azevedo
Rafael, Eurico Martins
Guilherme Santos, André
Bruno Machado, Daniel Mendes
Tiago Ferreira, Henrique Abrantes

Table 4: Lista de Membros

(Todas as referências a relatórios ou documentos entregues pelas respetivas equipas foram feitas)

References

- [25a] *Modelo de Cores HSV*. Baseado em aulas do professor Pedro Teodoro e caderno. 2025. URL: <https://pt.wikipedia.org/wiki/HSV>.
- [25b] *PID - Controlador Proporcional Integral Derivativo*. Baseado em aulas do professor Pedro Teodoro e caderno. 2025. URL: https://pt.wikipedia.org/wiki/Controlador_proporcional_integral_derivativo.
- [Abr25a] Henrique Abrantes. *Recensão crítica sobre "Survey on advances on terrain based navigation for autonomous underwater vehicles"*. Tech. rep. Recensão Crítica. Escola Náutica Infante Dom Henrique, 2025.
- [Abr25b] Henrique Abrantes. *Recolha de Dados1*. telemetria da recolha de dados do PID. 2025. URL: <https://github.com/Bolofofopt/UnderwaterDroneProject/tree/main/mydocs/telemetria/PID>.
- [Abr25c] Henrique Abrantes. *Recolha de Dados3*. telemetria da recolha de dados do algoritmo de visão. 2025. URL: <https://github.com/Bolofofopt/UnderwaterDroneProject/tree/main/mydocs/telemetria/visao>.
- [Abr25d] Henrique Abrantes. *Repositório GitHub*. 2025. URL: <https://github.com/Bolofofopt/UnderwaterDroneProject>.
- [Blu24a] BlueRobotics. *BlueOS Documentation*. Acesso em maio de 2025. 2024. URL: <https://docs.bluerobotics.com/blueos/>.
- [Blu24b] BlueRobotics. *BlueROV2 Maintenance Guide*. Acesso em maio de 2025. 2024. URL: <https://bluerobotics.com/learn/bluerov2-maintenance-checklist/>.
- [Blu24c] BlueRobotics. *BlueROV2 Technical Documentation*. Acesso em maio de 2025. 2024. URL: <https://bluerobotics.com/learn/bluerov2-technical-details/>.
- [Blu24d] BlueRobotics. *Pre-Dive Checklist for BlueROV2*. Acesso em maio de 2025. 2024. URL: <https://bluerobotics.com/learn/bluerov2-pre-dive-checklist/>.
- [Coi] Miguel Tavares Coimbra. *Capítulo III – Processamento de Imagem*. URL: https://www.dcc.fc.up.pt/~mcoimbra/lectures/PSI_1011/Aula%204%20-%20Capitulo%20III.pdf.
- [Con25] MAVLink Router Contributors. *MAVLink Router: Route MAVLink packets between endpoints*. A tool for routing MAVLink messages between multiple endpoints. 2025. URL: <https://github.com/mavlink-router/mavlink-router>.
- [Dan25] Francisco Azevedo Daniel Mendes João Stoffel. *Relatório do Projeto 'ROV'*. Equipa de Visão, TBN e Nav. Acústica. 2025.
- [Fra25] Diogo Soares e Francisca Grazina. *Controlador PID*. Equipa de Pesquisa. 2025.
- [Lia22] Oscar Liang. *LiPo Battery Safety Guide: Tips and Precautions*. Acesso em maio de 2025. 2022. URL: <https://oscarliang.com/lipo-battery-guide/>.
- [MM17] José Melo and Aníbal Matos. "Survey on advances on terrain based navigation for autonomous underwater vehicles". In: *Ocean Engineering* 139 (2017), pp. 250–264. DOI: 10.1016/j.oceaneng.2017.04.047.
- [Pro24] QGroundControl Project. *QGroundControl User Guide*. Acesso em maio de 2025. 2024. URL: <https://docs.qgroundcontrol.com/>.
- [Vin25] Vinicius. *Modelos de Cores HSV e LAB*. Acesso em maio de 2025. 2025. URL: <https://www.monolitonimbus.com.br/modelos-de-cores-hsv-e-lab/>.